Game Artificial Intelligence for Tetris

Yıldız İşleyen

January 2025

1 Introduction

Tetris is a puzzle game based on strategically placing differently shaped blocks within a limited play area. It features simple rules, but offers deep strategic complexity. The main objective of the game is to arrange the blocks without leaving gaps to complete horizontal rows and clear them to earn points.

Various artificial intelligence techniques have been developed to enable more effective and efficient decision-making in games like Tetris. In the first phase of this study, only the Greedy Algorithm which selects the best-looking action at each step, but it does not always find the optimal solution, was used; however, to achieve better results, Reinforcement Learning has been incorporated into this phase. The Q-learning algorithm will attempt to learn the optimal strategy by updating the rewards given to each action throughout the game. Additionally, the Greedy Algorithm will be used for comparison purposes.

2 Literature Review

When researching similar studies, there are some algorithms that are noticed and used in the majority. The first and most commonly used of these is the genetic algorithm. Genetic algorithms are a type of search and optimization method inspired by the process of natural selection. When comparing these two approaches, it is examined under 3 parts:

2.1 Orthogonal

The Q-learning algorithm is effective in adapting to dynamic environments and optimizing long-term strategies by learning rewards over time. Although it may take longer to converge and requires more computational resources, it can outperform other algorithms in decision-making quality during extended game-play. On the other hand, genetic algorithms also require significant computational power but improve strategies by imitating natural selection. However, they may not be as effective in real-time decision-making compared to Q-learning.

2.2 Complementary

Q-learning's ability to learn optimal strategies over time allows it to handle complex decision-making and adapt to different game states. Meanwhile, genetic algorithms can enhance these strategies by using evolutionary processes, improving long-term game-play performance. Combining both methods could create a powerful system, where Q-learning focuses on immediate optimization, and genetic algorithms refine and evolve the strategy for better results over time.

2.3 Contradictory

A Q-learning algorithm, which aims to maximize long-term rewards, might not always align with genetic algorithms that focus on short-term evolutionary progress. Using both together could create conflicts, where Q-learning focuses on the best immediate action in each situation, while the genetic algorithm looks for gradual improvement over time. If not carefully managed, this could lead to a less effective combination of both approaches.

3 Dataset

This project does not use an external dataset. Data is generated in real-time based on the game state. The dataset for this project is dynamic, consisting of the Tetris game board, the tetrominoes (I, O, T, L, J, S, Z), and the corresponding actions, rewards, and transitions.

Projects like Tetris typically use data collected during the game, such as the current state of the game, actions, and rewards, rather than relying on a fixed data set. This data helps train the model and guide the decision-making process.

4 Baseline

The baseline implementation is based on the Q-learning algorithm. The Q-learning agent is designed to learn optimal strategies by exploring and exploiting the game environment. The baseline focuses on training the agent to maximize the long-term reward, where each action taken by the agent (such as rotating or placing a tetromino) is evaluated based on its immediate outcome (lines cleared) and the long-term potential (future rewards).

5 Main Approach

The Q-learning algorithm is a reinforcement learning method that allows the agent to learn the optimal strategies. In this project, Q-learning allows the agent to learn how to play Tetris by interacting with the environment and updating

its strategies based on rewards. The Q-table is used to store and update the estimated values for each state action pair. It helps the agent make better decisions by learning from its past actions and experiences.

In a single step of the agent's decision-making process, the game starts with an empty board and a random tetromino, such as a piece of T-shaped. The agent can perform 40 possible actions, including rotating the piece and placing it in any of the 10 columns. Using the epsilon-greedy strategy, the agent chooses an action, like rotating the piece 90 degrees and placing it in column 3. After placing the piece, if it clears a line, the agent gets a positive reward, such as +10. The Q-table is then updated by adjusting the value for the state-action pair based on the reward and the expected future reward.

6 Evaluation Metric

There are 4 evaluation metrics in this project. These are cleared lines, total score, episode duration and average score over episodes. The total score is calculated as the sum of the lines cleared in each step. The average score for episodes is calculated by summing the scores for all episodes and dividing by the number of episodes. A longer game duration combined with a higher score indicates that the agent is making better decisions and learning more effectively.

7 Results and Analysis

The comparison between the Q-Learning Agent and the Greedy Strategy Agent is essential to evaluate the effectiveness of different decision-making approaches in dynamic environments like Tetris. The Q-Learning Agent learns from experience, adapts, and improves over time, while the Greedy Strategy Agent only follows the best-known action and does not adapt.

By comparing the two, it becomes clear which approach is more effective in different situations, such as short-term and long-term performance, and how each strategy addresses complex or changing environments. In the long term, the Q-learning agent is expected to outperform the greedy strategy because it can adapt and improve over time. With enough training, it can discover the most effective strategies and maximize its score.

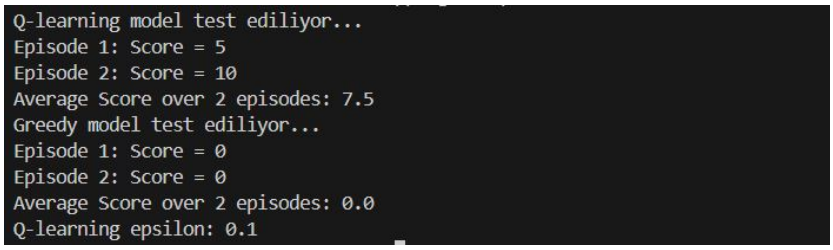


Figure 1: Q-learning vs Greedy

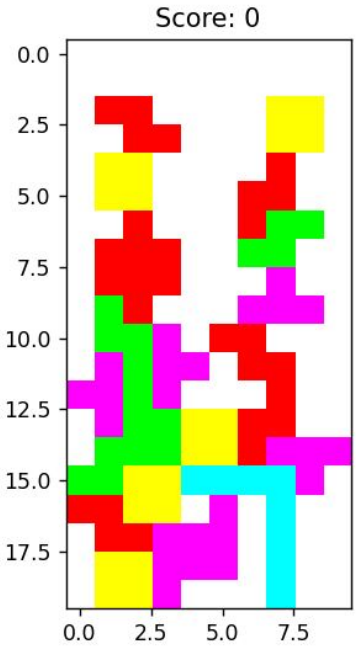


Figure 2: First Episode Output

8 Error Analysis

The model tends to learn as the number of episodes increases. It cannot give correct output for low number of episodes. Therefore, the greedy agent can give better results for some outputs. This leads to meaninglessness.

In addition, although the color of the Tetris blocks is fixed in the code, the colors can change while the model is learning.

9 Future Work

9.1 Deep Reinforcement Learning

Deep Reinforcement Learning can improve the Tetris AI by using neural networks to handle the large and complex state space. Instead of storing a Q-table, a Deep Q-Network can approximate the Q-values for state-action pairs.

9.2 Multiplayer Tetris AI

The model can be extended to compete against other AI agents or human players by introducing multiplayer functionality. This would involve designing strategies to not only maximize its own score but also disrupt the opponent's game-play,

such as sending garbage lines or blocking critical moves. Incorporating these competitive tactics would make the AI more dynamic and capable of adapting to diverse playing styles, enhancing its performance in multiplayer settings.

References

[1] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

[2] Russell, S. J., & Norvig, P. (2016). *Artificial Intelligence: A Modern Approach*. Pearson.

[3] Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3), 279-292.

[4] Zhu, X., & Iglewicz, B. (1998). Using Q-learning for an Autonomous Tetris Agent. *Proceedings of the International Conference on Artificial Intelligence and Games*.